EMAIL CAMPAIGN PLATFORM

Project Brief • Architecture • Technology Stack • Implementation Requirements

Purpose

A web application that imports client contacts from Excel, creates email campaigns, and sends one individually addressed email per recipient through a Google Workspace Gmail account. Email delivery is handled asynchronously using Cloudflare Queues and Cloudflare Workers.

Decision area	Proposed selection
Application language	TypeScript
Frontend	React + Vite
Backend runtime / API	Cloudflare Workers + Hono
Database	Cloudflare D1 (SQLite; SQL relational database)
Background processing	Cloudflare Queues + Queue Consumer Worker
File storage	Cloudflare R2
Email provider	Gmail API (Google Workspace)
Local development	Wrangler + local D1 / local bindings

Prepared: 6 November 2026 | Status: Initial technical specification for AI-assisted project scaffolding

Document Guide

• 1. Project overview and goals

• 2. Scope and key user workflows

• 3. Recommended technology stack

• 4. High-level architecture

• 5. Cloudflare services and hosting model

• 6. Database approach and proposed schema

• 7. Email sending integration

• 8. Background processing and reliability

• 9. Excel import and validation

• 10. Security, privacy and compliance

• 11. Local development and environments

• 12. Suggested repository structure

• 13. API surface (initial proposal)

• 14. Non-functional requirements and acceptance criteria

• 15. Implementation phases

• 16. Assumptions, open decisions and guidance for Claude

Important architecture note

This specification deliberately selects Cloudflare D1 rather than PostgreSQL. D1 is a managed, SQLite-based relational database and uses SQL. It is not Microsoft SQL Server or PostgreSQL. The project should use SQLite-compatible schema definitions and queries through Drizzle ORM.

1. Project Overview and Goals

Build a secure web application for importing a list of client contacts from an Excel workbook and sending a separate email to each selected recipient. The application is intended initially for internal business outreach and campaign management. It should be designed cleanly enough to support future enhancements such as multiple users, reusable templates, scheduling, and multi-account sending, without overengineering the first release.

Primary goals

- Import .xlsx contact lists and map the email address column (and optionally client name and other fields).
- Validate, normalize, and deduplicate email addresses before campaign creation.
- Create campaigns with a name, subject, HTML body, sender account, and selected recipients.
- Send each recipient a separate message; do not expose other recipients in To, CC, or BCC.
- Process sending asynchronously so a browser request does not remain open while emails are sent.
- Track queued, processing, accepted/sent, failed, skipped, and retry states.
- Provide a dashboard and campaign detail view with progress and error reporting.

Out of scope for the first release

- Full marketing automation, lead scoring, CRM functionality, and complex customer segmentation.
- Guaranteed inbox delivery. Gmail API acceptance is not proof that a message reached the inbox.
- Large-scale high-volume marketing delivery. Gmail Workspace sending limits apply; use a dedicated email delivery provider if volume grows substantially.
- Advanced drag-and-drop email design unless added as a later feature.

2. Scope and Key User Workflows

A. Import contacts

- Authenticated user selects an Excel workbook and uploads it through the application.
- Backend stores the original file in R2 and creates an import record.
- Parse the workbook, identify sheets and headers, and allow the user to select the relevant worksheet and map columns.
- At minimum, require an email column. Client name is optional but recommended for personalization.
- Show a preview and import summary: total rows, valid rows, invalid addresses, duplicates, and blank email values.
- Persist accepted contacts and import validation results. Do not silently discard invalid rows; make them reviewable or downloadable.

B. Create and launch campaign

- User creates a campaign and supplies campaign name, subject, HTML body, and optional plain-text body.
- User selects imported recipients (initial release may use all valid contacts from one import).
- Provide a preview and a test-send action before launch.
- On confirmation, create recipient-level send records and enqueue jobs in manageable batches.
- Campaign status progresses through Draft, Queued, Sending, Completed, CompletedWithErrors, Failed, or Cancelled.

C. Monitor and recover

- Campaign dashboard displays recipient counts by status and progress.
- Campaign detail shows per-recipient status, timestamps, attempts, and sanitized error information.
- Allow retry of eligible failed messages and pause/cancel of campaigns where feasible.
- Record every attempt in an email log. Ensure a retry does not accidentally create uncontrolled duplicate sends.

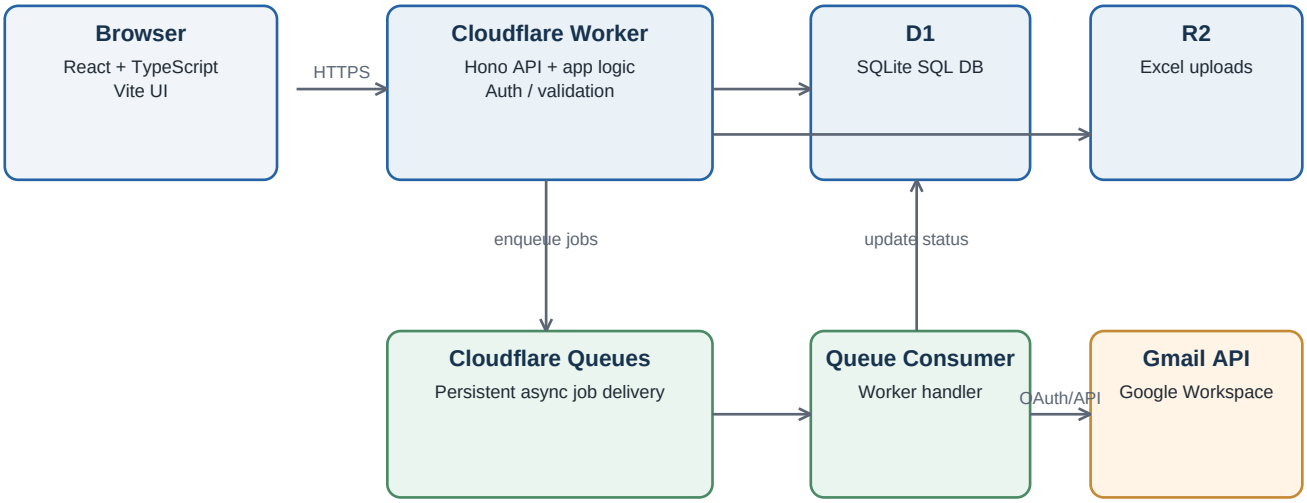
3. Recommended Technology Stack

Layer	Technology	Reason / responsibility
Language	TypeScript	Shared language across frontend, API, and queue consumer.
Frontend	React + Vite	Responsive single-page application; build assets for Workers static assets.
UI styling	Tailwind CSS	Consistent, responsive UI styling.
API runtime	Cloudflare Workers	Serverless application runtime; no VM or always-on server.
API framework	Hono	Lightweight routing, middleware, request validation integration.
Database	Cloudflare D1	Managed SQLite-based relational SQL database.
ORM / migrations	Drizzle ORM + Drizzle Kit	Typed schema, query builder, migration generation.
Async jobs	Cloudflare Queues	Durable asynchronous message processing and retry controls.
Object storage	Cloudflare R2	Original Excel files and optional generated exports.
Excel parser	SheetJS (xlsx)	Read workbook sheets, headers, and cell values.
Email provider	Gmail API	Send as the connected Google Workspace mailbox using OAuth 2.0.
Validation	Zod	Validate API payloads, environment settings, and parsed data.
Testing	Vitest	Unit and service-level tests; add integration tests with Wrangler test utilities.
Local tooling	Wrangler	Run Workers locally and simulate D1, Queues, and R2 bindings.
Source control / CI	GitHub + GitHub Actions	Version control and automated checks/deployment.

Use current stable versions at implementation time. Verify Cloudflare runtime compatibility for every package; Node.js-only libraries may require unsupported APIs or compatibility flags.

4. High-Level Architecture

The browser communicates with a Cloudflare Worker API built with Hono. The API validates requests and coordinates D1, R2, and Queues. A separate Queue Consumer handler (which may be in the same Worker codebase/deployment configuration) processes individual email jobs. The consumer calls the Gmail API and persists outcomes in D1.



Cloudflare services run in the Cloudflare account; Gmail API is an external Google Workspace service.

Main request flow

- Browser → Worker API: upload, import preview, campaign management, launch, monitoring.
- Worker API → R2: store uploaded workbook (prefer direct or streamed upload for larger files).
- Worker API → D1: store import metadata, recipients, campaign, recipient-level email jobs.
- Worker API → Cloudflare Queues: publish job messages after durable database records exist.
- Queue Consumer → Gmail API: send one message to one recipient using OAuth credentials.
- Queue Consumer → D1: record result, attempt count, timestamps, and sanitized error.
- Browser → Worker API → D1: query campaign metrics and recipient statuses.

Important design choice: D1 is the system of record for campaigns and recipient status. Cloudflare Queues is the transport mechanism for asynchronous work, not the only record of work. Persist jobs/status in D1 before publishing queue messages, and design recovery for the small failure window between the database write and queue publication.

5. Cloudflare Services and Hosting Model

Service	Use in this project	Notes
Workers	Serve API and application logic; host built frontend assets where appropriate.	No continuously running server process.
D1	Store application data and campaign state.	SQLite dialect; SQL queries; managed by Cloudflare.
Queues	Dispatch recipient-level email jobs to a consumer.	Configure batch size, retry policy, and dead-letter queue (DLQ).
R2	Store uploaded Excel workbooks and downloadable reports.	Private bucket; use Worker-mediated access or short-lived access patterns.
Secrets / vars	Store OAuth client secret, refresh token, and other sensitive configuration.	Use Wrangler secrets in deployed environments; never commit secrets.
Cron Triggers (optional)	Schedule campaigns or run reconciliation/cleanup tasks.	Use only if scheduled sending or periodic recovery is required.
Workflows (optional)	Orchestrate complex, multi-step long-running processes.	Not required for the MVP; Queues are sufficient for recipient-level jobs.

Hosting model

The application is deployed to Cloudflare Workers. D1, R2, and Queues are provisioned in the Cloudflare account and bound to the Worker through Wrangler configuration. Gmail API remains an external Google service. The application does not need a traditional VPS, Windows service, container host, or separate always-on background server.

For a new full-stack Cloudflare project, use Workers with static assets rather than assuming Cloudflare Pages is required. Keep frontend and API deployable together initially; split them later only if there is a concrete operational reason.

6. Database Approach and Proposed Schema

Database: Cloudflare D1. D1 is a managed relational database built on SQLite. It supports SQL and relational tables, indexes, constraints, and transactions within D1's supported feature set. It is not PostgreSQL and not Microsoft SQL Server. Use Drizzle's SQLite schema package (drizzle-orm/sqlite-core), not PostgreSQL schema definitions.

Table	Purpose	Representative fields
users	Application identities (if app-managed auth is used).	id, email, display_name, role, created_at
imports	Track workbook uploads and parsing results.	id, file_key, original_filename, status, row_count, created_by, created_at
import_rows (optional)	Retain row-level validation results for review.	id, import_id, row_number, raw_email, normalized_email, validation_status, error
contacts	Reusable normalized client/contact records.	id, email, normalized_email, name, source_import_id, consent_status, created_at
campaigns	Campaign configuration and aggregate state.	id, name, subject, html_body, text_body, status, scheduled_at, created_by, timestamps
campaign_recipients	Recipient-level campaign work and status.	id, campaign_id, contact_id, email_snapshot, status, attempt_count, queued_at, sent_at, last_error
email_attempts	Audit trail for every send attempt.	id, campaign_recipient_id, attempt_number, outcome, provider_message_id, error_code, error_message, attempted_at
email_templates (optional)	Reusable campaign templates.	id, name, subject_template, html_body, text_body, created_at
oauth_accounts	OAuth token metadata, if persisted in D1.	id, provider, mailbox_email, encrypted_token_payload, scopes, updated_at

Schema and data rules

- Use UUID or similarly collision-resistant IDs; store timestamps consistently (UTC Unix milliseconds or ISO-8601 text, chosen consistently).
- Store both original and normalized email values. Normalize whitespace and case for comparison; do not attempt aggressive transformations such as removing Gmail dots.
- Add indexes for campaign_id/status, normalized_email, and import_id where query patterns require them.
- Use unique constraints where appropriate, such as one campaign-recipient row per campaign/contact, subject to product rules.
- Store an email address snapshot on campaign_recipients so historical campaigns remain auditable if a contact is later edited.
- Do not store raw OAuth tokens unencrypted. Prefer encrypted token storage with a key held as a Worker secret, or another secure token-storage approach.

7. Email Sending Integration

Use the Gmail API to send messages from the user's Google Workspace mailbox. Authenticate using Google OAuth 2.0. The application should request only the scopes needed for sending, typically the Gmail send scope. OAuth consent-screen configuration, Workspace administrator policies, and Google API project configuration must be completed before production use.

Recommended flow

- An administrator connects the Google Workspace mailbox through an OAuth authorization flow.
- Worker handles the OAuth callback and securely stores the refresh token (encrypted at rest); access tokens are refreshed as required.
- Queue Consumer builds a MIME email with To, Subject, MIME-Version, Content-Type, and body parts, then base64url-encodes it for Gmail API messages.send.
- Send exactly one recipient per message. Personalize the greeting using validated contact fields when configured.
- Persist Gmail API response message ID and API acceptance timestamp. Treat API errors separately from delivery/bounce status.
- Implement provider error classification: retry transient network/5xx/rate-limit errors with backoff; do not repeatedly retry permanent invalid-recipient or permission errors.

Sender identity and Gmail limitations

The From identity must be the authenticated Workspace mailbox or a properly configured Gmail send-as alias. Gmail Workspace has per-account sending and recipient limits that vary by account type and policy and can change. Read the current official Google Workspace sending limits before launch; do not hard-code an assumed universal daily quota. Add configurable conservative per-minute and daily limits, and stop/hold the campaign when the configured quota is reached.

A successful Gmail API response means Google accepted the message for processing; it does not guarantee inbox placement or final delivery. The MVP should label this state 'Accepted/Sent' rather than 'Delivered' unless a separate delivery/bounce signal is integrated.

8. Background Processing and Reliability

Do not send all campaign emails inside the HTTP request that launches a campaign. The launch endpoint should validate the campaign, persist recipient jobs, and enqueue work. The Queue Consumer processes jobs independently.

Suggested message payload

```
{
  "campaignRecipientId": "uuid",
  "campaignId": "uuid",
  "idempotencyKey": "campaign-recipient-uuid"
}
```

Queue consumer responsibilities

- Validate message payload and load the recipient/campaign data from D1.
- Atomically claim a recipient job or check its current status before sending.
- Enforce account-level rate limits and bounded concurrency.
- Send the email and persist the provider response.
- On transient failure, use bounded exponential backoff and queue retry policy.
- After maximum attempts, mark the job failed and/or route the message to a dead-letter queue for inspection.
- Provide reconciliation logic for records stuck in Processing or persisted-but-not-enqueued states.

Idempotency and duplicate protection

Queue systems may deliver a message more than once. Design for at-least-once processing: use a stable idempotency key, claim/lease state in D1, and a clear state machine. There is an unavoidable edge case if Gmail accepts a message but the Worker crashes before recording success; the application should document and mitigate this uncertainty rather than promise exactly-once email delivery.

Suggested status values

Entity	Statuses (suggested)
Campaign	Draft, Queuing, Queued, Sending, Paused, Completed, CompletedWithErrors, Failed, Cancelled
Campaign recipient	Pending, Queued, Processing, Accepted, Failed, Skipped, Cancelled

9. Excel Import and Validation

Accept .xlsx files initially. Set a reasonable configurable file-size and row-count limit. Treat uploaded workbooks as untrusted input.

- Validate extension and file signature/content; do not rely only on the filename or browser MIME type.
- Set file-size, worksheet-count, and row-count limits to prevent resource exhaustion.
- Allow worksheet and column selection where possible; do not assume the email column is always named 'Email'.
- Trim leading/trailing whitespace and validate address syntax. Syntax validation does not prove that a mailbox exists.
- Report blank, malformed, and duplicate email values with source row numbers.
- Support UTF-8 names and common Excel cell formats; avoid evaluating formulas or macros.
- Store the original file in a private R2 bucket only as long as required by retention policy.

Personalization

Support a controlled set of placeholders such as {{firstName}} and {{companyName}}. Escape values inserted into HTML to prevent HTML/script injection. Provide a preview and test-send workflow. Do not allow arbitrary user-supplied template code to execute.

10. Security, Privacy and Compliance

- Require authentication for all application routes except explicitly public OAuth callback/health endpoints.
- Use role-based authorization for campaign creation, sending, account configuration, and user administration.
- Protect OAuth credentials and refresh tokens; use Worker secrets for client secrets and encryption keys. Never put secrets in frontend code, source control, logs, or PDF/config examples.
- Use HTTPS, validate inputs, apply upload limits, and protect against CSRF where cookie-based sessions are used.
- Restrict R2 bucket access; do not expose uploaded workbooks through public URLs.
- Log operational metadata, not full email bodies, OAuth tokens, or unnecessary personal data.
- Support consent/source tracking and suppression/unsubscribe lists for marketing communications. Check applicable laws including privacy and anti-spam requirements in the jurisdictions of the sender and recipients.
- Provide a way to suppress opted-out, bounced, complained-about, or otherwise ineligible recipients before sending.
- Define data retention and deletion procedures for imported files, contacts, campaign content, and logs.

Authentication note

The final authentication provider is an implementation decision. For a private internal tool, Cloudflare Access may be the simplest initial gate if the intended users are within a controlled organization. If the product needs application-managed users and roles, select a Cloudflare-compatible authentication library and verify its D1/session support before scaffolding. Do not assume a specific auth library without validating current compatibility.

11. Local Development and Environments

The complete application can be developed and tested on a local Windows machine. Wrangler runs the Worker locally and provides local simulations for D1, R2, and Queues. Local D1 data is separate from the remote Cloudflare database.

Environment	Services	Purpose
Local	Vite dev server, Wrangler dev, local D1/R2/Queues simulation, Google test mailbox/API project	Daily coding and automated tests.
Staging	Separate Cloudflare Worker, D1 database, R2 bucket, Queue and OAuth app/config	End-to-end integration and deployment verification.
Production	Production Worker, D1, R2, Queue, approved Workspace mailbox	Live use.

Local workflow

```
# Install dependencies
npm install

# Run frontend / Worker dev command(s) defined by the project
npm run dev

# Generate Drizzle migrations
npx drizzle-kit generate

# Apply migrations to local D1
npx wrangler d1 migrations apply <database-name> --local

# Apply migrations to the remote/staging D1 database
npx wrangler d1 migrations apply <database-name> --remote
```

Exact commands and scripts must be finalized in package.json and wrangler.jsonc during scaffolding. Local Queue behavior is a development simulation; validate retry, concurrency, and failure behavior in a real Cloudflare staging environment before production.

Never point local development at production D1 by default. Use separate bindings and credentials for local, staging, and production.

12. Suggested Repository Structure

```
email-campaign-platform/
■■■ src/
■   ■■■ client/                # React application
■   ■   ■■■ components/
■   ■   ■■■ pages/
■   ■   ■■■ hooks/
■   ■   ■■■ services/          # Typed API client
■   ■■■ server/
■   ■   ■■■ app.ts             # Hono app / route registration
■   ■   ■■■ routes/
■   ■   ■■■ modules/
■   ■   ■   ■■■ campaigns/
■   ■   ■   ■■■ contacts/
■   ■   ■   ■■■ imports/
■   ■   ■   ■■■ email/
■   ■   ■   ■■■ accounts/
■   ■   ■■■ queue/
■   ■   ■   ■■■ producer.ts
■   ■   ■   ■■■ consumer.ts
■   ■   ■■■ database/
■   ■   ■   ■■■ schema/
■   ■   ■   ■■■ repositories/
■   ■   ■■■ middleware/
■   ■   ■■■ services/
■   ■■■ shared/                # Shared types / schemas
■■■ migrations/
■■■ tests/
■■■ public/
■■■ drizzle.config.ts
■■■ wrangler.jsonc
■■■ vite.config.ts
■■■ package.json
■■■ tsconfig.json
■■■ README.md
```

Use clear module boundaries: routes handle HTTP concerns; services implement use cases; repositories encapsulate D1/Drizzle access; providers encapsulate Gmail API calls; queue handlers orchestrate asynchronous processing. Keep Cloudflare-specific bindings behind interfaces where practical to make unit tests easier.

13. Initial API Surface (Proposal)

Method + path	Purpose
POST /api/imports	Create an import and upload/parse workbook (or create upload session).
GET /api/imports/:id	Get import status and summary.
GET /api/imports/:id/rows	Review invalid/duplicate rows with pagination.
POST /api/imports/:id/contacts	Confirm and persist valid contacts from import.
GET /api/contacts	List/search imported contacts.
POST /api/campaigns	Create draft campaign.
GET /api/campaigns	List campaigns.
GET /api/campaigns/:id	Get campaign detail and aggregate counts.
PATCH /api/campaigns/:id	Update a draft campaign.
POST /api/campaigns/:id/test-send	Send a test message to an authorized test address.
POST /api/campaigns/:id/launch	Validate, create recipient jobs, and enqueue campaign.
POST /api/campaigns/:id/pause	Pause future processing where supported.
POST /api/campaigns/:id/cancel	Cancel unsent recipient jobs where supported.
POST /api/campaigns/:id/retry-failed	Requeue eligible failed recipients.
GET /api/campaigns/:id/recipients	Paginated recipient statuses and errors.
GET /api/account/email	Show connected sender mailbox status.
GET /api/health	Health check; expose no secrets.

All list endpoints should support pagination. Validate request/response schemas and return consistent error envelopes. Add API versioning if external clients are introduced.

14. Non-Functional Requirements and Acceptance Criteria

- Reliability: a campaign launch must not lose recipient work if an HTTP request times out after jobs are persisted.
- Scalability: process queue messages in bounded batches and respect Gmail quota/rate limits.
- Observability: structured logs with campaign ID, recipient-job ID, attempt number, and correlation ID; no secrets or unnecessary message content.
- Usability: user can preview import results, test an email, launch a campaign, and see per-recipient outcomes.
- Security: unauthorized users cannot view contacts, campaign content, imports, or send emails.
- Maintainability: strict TypeScript, linting, formatting, typed environment bindings, migrations, and automated tests.
- Accessibility: forms, tables, status labels, and navigation should be keyboard-accessible.

MVP acceptance checklist

- User can upload an .xlsx file and map the email column.
- System reports valid, invalid, blank, and duplicate rows before confirmation.
- User can create a draft campaign and preview its rendered email.

- User can send a test email through the connected Workspace account.
- Launching a campaign creates one independent recipient job per eligible contact.
- Queue Consumer sends individual emails and records Gmail API acceptance or failure.
- Dashboard accurately reflects persisted recipient states and attempt counts.
- Transient failures retry within configured limits; permanent failures are not endlessly retried.
- Local development works with local D1 and simulated Cloudflare bindings; staging verifies real queue and Gmail integration.

15. Implementation Phases

Phase	Deliverables
0. Foundation	Repository, strict TypeScript, Vite/React, Hono Worker, Wrangler config, environment bindings, lint/test scripts.
1. Data layer	D1 database, Drizzle schema and migrations, repositories, seed/test utilities.
2. Contacts import	R2 upload, Excel parsing, column mapping, validation preview, deduplication, import persistence.
3. Campaign management	Campaign CRUD, HTML/plain-text content, recipient selection, preview, test send.
4. Gmail integration	Google Cloud project/OAuth setup, token lifecycle, Gmail API provider, safe secret handling.
5. Queue processing	Queue producer/consumer, rate limiting, retries, DLQ, idempotency, reconciliation.
6. Dashboard	Campaign list/detail, progress metrics, recipient-level outcomes, retry controls.
7. Hardening & deploy	Authentication/authorization, compliance controls, tests, staging deployment, monitoring, production rollout.

16. Assumptions, Open Decisions and Guidance for Claude

Current assumptions

- Initial use is an internal application for business outreach, not a public multi-tenant SaaS product.
- One Google Workspace mailbox is connected initially; design provider/account abstractions so additional mailboxes could be added later.
- One recipient receives one separate email. No bulk CC/BCC behavior.
- Cloudflare D1 is the selected database. Do not substitute PostgreSQL unless the project owner explicitly changes this decision.
- Cloudflare Workers, D1, Queues, and R2 are the hosting and infrastructure foundation.
- Credentials will be supplied later. Scaffold configuration placeholders and document setup steps; do not invent, request, or commit credentials.
- Use a dedicated Google Cloud project and OAuth client for the application. Production OAuth verification and Workspace policies may affect deployment.

Decisions to confirm during implementation

- Whether the application is single-user/internal or requires multiple application users and role management.
- Whether Cloudflare Access or app-managed authentication is preferred.
- Expected campaign size and desired sending cadence; configure conservative defaults until confirmed.
- Whether contacts should be globally reusable across imports or scoped to an import/list.
- Whether unsubscribe/suppression management is required in the first release (recommended for marketing use).
- Whether scheduled campaigns are needed in the MVP or can be deferred.

Instructions for Claude / scaffolding agent

- First convert this brief into a detailed implementation plan and a project README.md. Identify assumptions and unresolved decisions explicitly.
- Then scaffold the project in small, verifiable phases. Do not generate the entire application as one unreviewable code dump.
- Use TypeScript throughout, strict compiler settings, Hono for API routing, React + Vite for the UI, Drizzle SQLite schema for D1, and Wrangler for Cloudflare development/deployment.
- Use Cloudflare D1, R2, and Queues bindings. Include local development configuration and separate local/staging/production settings.
- Create database migrations and typed repositories. Do not use PostgreSQL-specific SQL or packages.
- Implement a Gmail provider abstraction and OAuth flow with placeholder environment variables. Do not hard-code credentials or assume tokens are available.
- Implement queue processing with bounded concurrency, configurable rate limits, retries, dead-letter handling, idempotency protections, and persisted status transitions.
- Write tests for validation, campaign state transitions, import parsing, queue message handling, and Gmail provider error classification.
- Document Cloudflare resource provisioning, Google OAuth setup, secrets configuration, local commands, migration commands, staging deployment, and production deployment.
- Before implementing ambiguous security/authentication decisions, present the options and recommend a safe default rather than silently choosing.

Core architecture decision

React + TypeScript on the frontend; Cloudflare Workers + Hono for the backend; Cloudflare D1 (SQLite/SQL) for relational data; Cloudflare Queues for asynchronous email jobs; Cloudflare R2 for uploaded workbooks; Gmail API + Google OAuth 2.0 for sending through Google Workspace.

Reference documentation

Cloudflare Workers: developers.cloudflare.com/workers/
Cloudflare D1: developers.cloudflare.com/d1/
Cloudflare Queues: developers.cloudflare.com/queues/
Cloudflare R2: developers.cloudflare.com/r2/
Wrangler: developers.cloudflare.com/workers/wrangler/
Hono: hono.dev
Drizzle ORM: orm.drizzle.team
Gmail API sending: developers.google.com/workspace/gmail/api/guides/sending
Google Workspace sending limits: support.google.com/a/answer/166852

This document is an implementation brief, not a guarantee of current service limits or package compatibility. Verify current Cloudflare and Google documentation before production deployment.

17. Suggested Application Features

This feature catalog expands the functional scope beyond the core import-and-send workflow. It is intended to guide Claude when producing the requirements Markdown file and scaffolding. Features are grouped into an MVP baseline and later enhancements; avoid implementing every enhancement before the core workflow is stable.

17.1 MVP features

Feature area	Suggested functionality	Priority
Dashboard	Campaign totals; draft/queued/sending/completed/failed counts; recipient totals by status; recent campaign activity.	MVP
Excel import	Upload .xlsx; select worksheet; map email/name/company columns; preview rows; validate; deduplicate; show import summary; download invalid-row report.	MVP
Contact management	Search, filter, view and edit contact details; identify source import; preserve normalized email; prevent accidental duplicates.	MVP
Campaign management	Create, edit, duplicate and archive drafts; campaign name, subject, HTML and plain-text body; recipient selection; status and timestamps.	MVP
Email preview	Preview rendered HTML; show personalization substitutions; send a test email to an authorized address before launch.	MVP
Individual sending	One recipient per Gmail API message; optional greeting personalization; sender mailbox identity; no shared To/CC/BCC list.	MVP
Queue processing	Queue status visibility; bounded concurrency; account-level rate limits; retries with backoff; dead-letter handling; idempotency safeguards.	MVP
Campaign monitoring	Progress summary; recipient-level status; attempt count; last error; provider message ID; sent/accepted timestamp.	MVP
Failure recovery	Retry eligible failed recipients; do not retry permanent errors endlessly; preserve an audit trail of every attempt.	MVP
Settings	Connected Google Workspace mailbox status; OAuth connection/reconnection flow; configurable safe sending limits; environment health.	MVP
Security	Authentication; role/permission checks; private R2 objects; secret management; audit events for campaign launch and account changes.	MVP

17.2 Recommended enhancements after MVP

Feature area	Suggested functionality	Notes
Reusable templates	Create, edit, preview, clone and archive reusable HTML/plain-text templates.	Include placeholder support and HTML escaping.
Personalization	Use approved merge fields such as {{firstName}}, {{companyName}} and custom mapped columns.	Preview missing values and define safe fallback text.
Scheduling	Schedule a campaign for a future date/time; timezone selection; scheduled/ready/paused states.	Use Cron Triggers or a suitable scheduling pattern.
Pause / resume / cancel	Pause further queue consumption for a campaign, resume it, or cancel unsent recipients.	Already accepted emails cannot be recalled.
Suppression / unsubscribe	Global suppression list; unsubscribe link and preference handling; suppress opted-out or ineligible recipients.	Strongly recommended for marketing outreach and compliance.
Segmentation	Filter contacts by import, company, tags, source, or custom fields; save recipient lists.	Avoid complex CRM features initially.
Attachments	Attach approved documents or campaign assets.	Enforce file-size/type limits and safe R2 access.
Exports	Export campaign results, recipient statuses, failed rows, and campaign summaries to CSV/XLSX.	Protect exported personal data.

Feature area	Suggested functionality	Notes
Multiple sender accounts	Connect and select from multiple authorized Workspace mailboxes.	Each mailbox needs its own OAuth authorization and quota controls.
Audit and activity history	Record who imported contacts, edited a campaign, launched it, retried jobs, or changed settings.	Use retention rules and avoid logging secrets.
Analytics	Campaign comparisons, sending duration, acceptance/failure rates, and trends over time.	Do not label API acceptance as inbox delivery.
Notifications	Notify an administrator when a campaign completes, stalls, or exceeds failure thresholds.	Email or in-app notifications.

17.3 Suggested screens / navigation

Screen	Primary content and actions
Overview	KPI cards, recent campaigns, queue health, alerts, quick actions.
Imports	Upload workbook, select sheet, map columns, preview validation, confirm import, view import history.
Contacts	Search and filter contacts, view/edit contact, see source import and suppression status.
Campaigns	Campaign list, status filters, create campaign, duplicate draft, archive.
Campaign editor	Campaign details, recipient selection, subject/body editor, merge-field helper, preview, test-send, launch confirmation.
Campaign details	Progress summary, recipient table, status filters, retry failed, pause/resume/cancel where applicable, export results.
Templates	Manage reusable email templates and preview them.
Settings	Google Workspace connection, sender identity, sending limits, user access, data-retention settings.

17.4 Suggested user roles

Role	Permissions
Administrator	Manage users/settings/sender connection; view all data; create, launch, pause, cancel and retry campaigns.
Campaign Manager	Import contacts; manage contacts and campaigns; preview/test-send; launch and monitor campaigns, subject to policy.
Viewer	Read dashboard, campaign status and reports; cannot import, edit, or send.

17.5 Feature rules and product behavior

- Require an explicit confirmation step before launching a campaign. Display the recipient count, sender mailbox, subject, and applicable sending-limit warning.
- Prevent launch if the sender account is disconnected, the campaign has no eligible recipients, the campaign content is invalid, or required compliance settings are incomplete.
- Support a test-send mode that cannot accidentally enqueue the full recipient list.
- Exclude suppressed/unsubscribed contacts from recipient selection and re-check suppression immediately before sending.
- Use pagination for contact and recipient tables; avoid loading an entire large campaign into the browser.
- Show clear distinction between Pending, Queued, Processing, Accepted by Gmail, and Failed. Do not claim delivery unless delivery evidence is available.
- Make retry actions explicit and auditable. Retry only transient or manually approved failures.
- Provide configurable retention and deletion for uploaded Excel files and personal contact data.

17.6 Implementation guidance

Build the MVP features first: import and validate contacts, create and preview a campaign, test-send, launch individual recipient jobs, process through Cloudflare Queues, and monitor outcomes. Keep later enhancements behind clean module boundaries. Claude should update the requirements README and implementation plan to include this feature catalog, identify

dependencies, and mark each feature as MVP or post-MVP before generating code.